

Módulo 5

Adrián Tobar Nicolau



Universitat
de les Illes Balears

MC

**Microcredencials
universitàries**
UIB

PLA
MICRO
CREDS



Financiado por
la Unión Europea
NextGenerationEU



Plan de Recuperación,
Transformación
y Resiliencia



Conselleria d'Educació
i Universitats
Direcció General d'Universitats, Recerca
i Ensenyament Superior

Guía para ggplot2

Guía para ggplot2

El paquete ggplot2 permite la creación de figuras dando un control total sobre sus propiedades. En esta sesión vamos a entrar en como crear, colorear, escalar y guardar una figura de ggplot2 a nuestro gusto.

Antes de empezar

Antes de empezar

Cargamos los paquetes que usaremos.

```
library(tidyverse)  
library(MASS)
```

Preparamos los datos de ejemplo.

```
data("birthwt")
bwt <- birthwt |>
  mutate(
    smoke = factor(smoke, labels = c("No fumadora",
                                      "Fumadora")),
    ht     = factor(ht, labels = c("No HTA",
                                    "HTA")),
    ui     = factor(ui, labels = c("No irritabilidad",
                                    "Irritabilidad")),
    race   = factor(race, labels = c("Blanca",
                                      "Negra",
                                      "Otra"))
  )
#fijamos peso a kg
bwt$lwt= 0.45359237*bwt$lwt
```

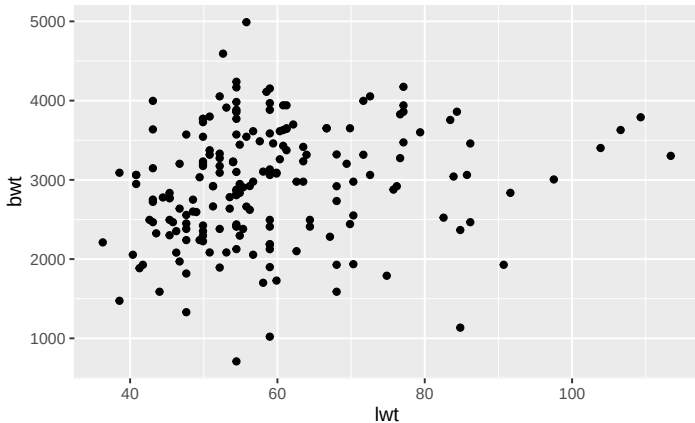
Datos, estética, y geometría

Datos, estética, y geometría

Toda figura de `ggplot2` tiene datos, estética y geometría.

Veamos un ejemplo básico.

```
ggplot(bwt, aes(x = lwt, y = bwt)) + geom_point()
```



Este gráfico de puntos está definido por

1. Datos: bwt.
2. Aes (estética): en el eje x el peso de la madre en su último período menstrual y en el eje y el peso del neonato en gramos.
3. Geometría: gráfico de puntos.

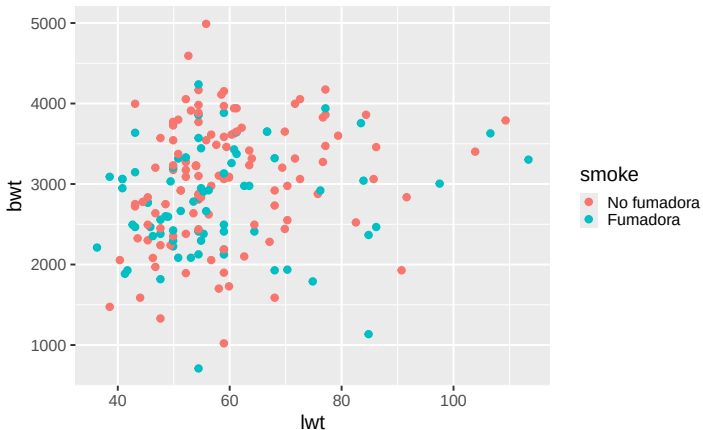
Habitualmente tendremos dos datos, uno para el eje x y otro para el eje y . Por comodidad podemos obviar la notación $x =, y =$ de la siguiente manera

```
ggplot(bwt, aes(lwt,bwt)) + geom_point()
```

Color

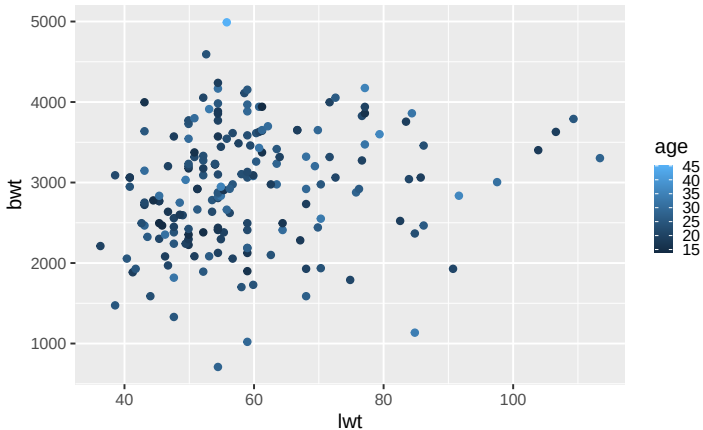
El paquete ggplot2 permite colorear la figura en función de datos

```
ggplot(bwt, aes(lwt,bwt, color=smoke)) + geom_point()
```



Además, detecta si usar un gradiente o categorías.

```
ggplot(bwt, aes(lwt,bwt, color=age)) + geom_point()
```

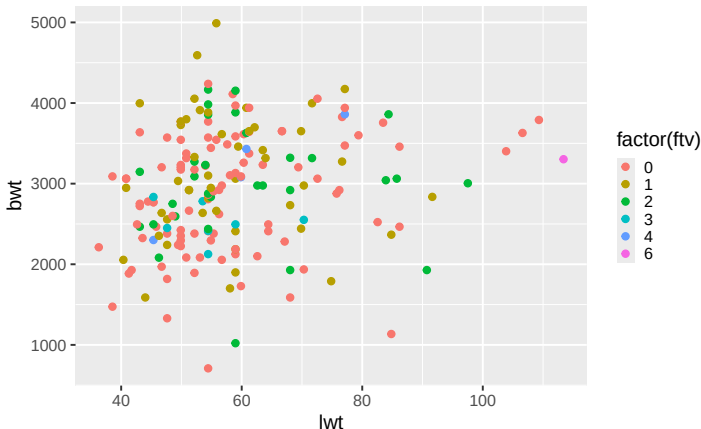


Las paletas básicas de ggplot2

La librería ggplot2 incluye varias paletas de colores entre las que elegir.

Ejemplo categórico por defecto

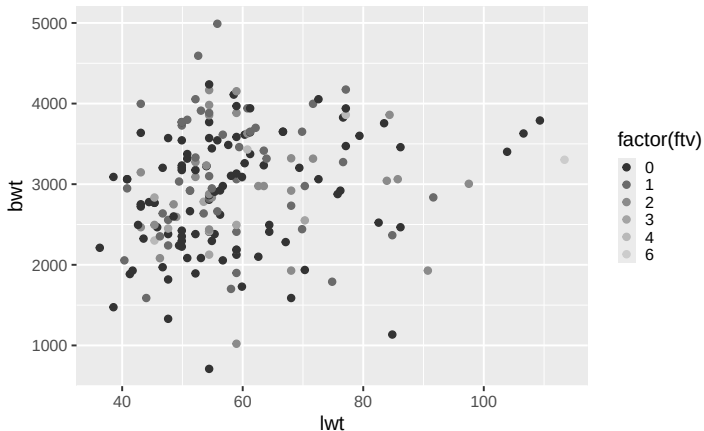
```
ggplot(bwt, aes(lwt,bwt, color=factor(ftv))) +  
  geom_point() + scale_colour_hue()
```



OJO: para que ggplot2 entienda una categoría como discreta tiene que estar como factor y para que sea continua no.

Ejemplo grises por defecto

```
ggplot(bwt, aes(lwt,bwt, color=factor(ftv))) +  
  geom_point() + scale_colour_grey()
```



Paletas ColorBrewer

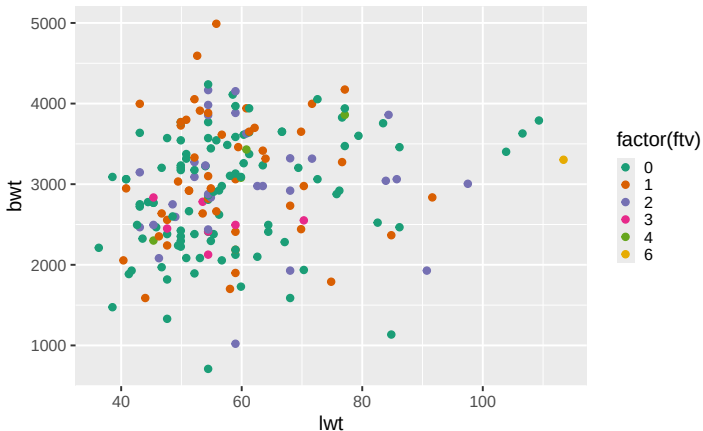
Para variables categóricas.

```
RColorBrewer::display.brewer.all(type = "qual")
```



Ejemplo categórica

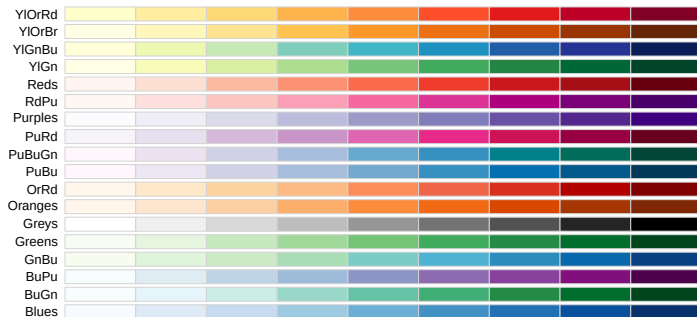
```
ggplot(bwt, aes(lwt,bwt, color=factor(ftv))) +  
  geom_point() + scale_colour_brewer(palette = "Dark2")
```



Paletas ColorBrewer

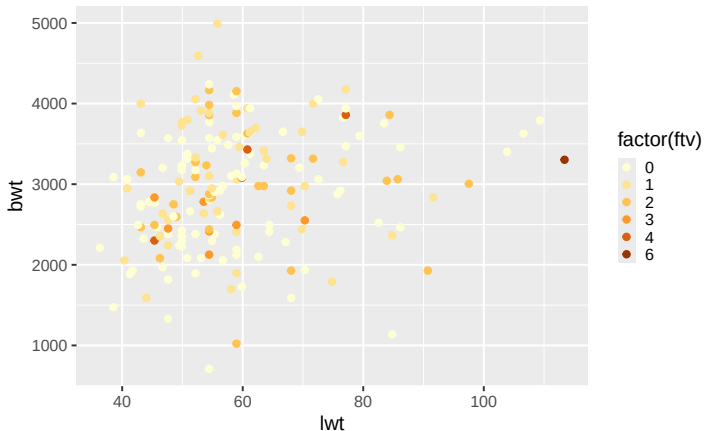
Para datos secuenciales.

```
RColorBrewer::display.brewer.all(type = "seq")
```



Ejemplo con YlOrBr

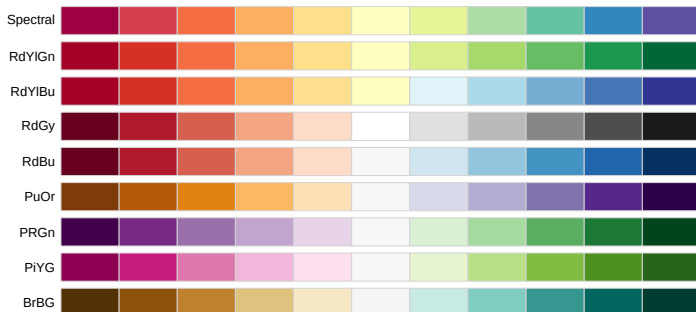
```
ggplot(bwt, aes(lwt,bwt, color=factor(ftv))) +  
  geom_point()+scale_colour_brewer(palette = "YlOrBr")
```



Paletas ColorBrewer

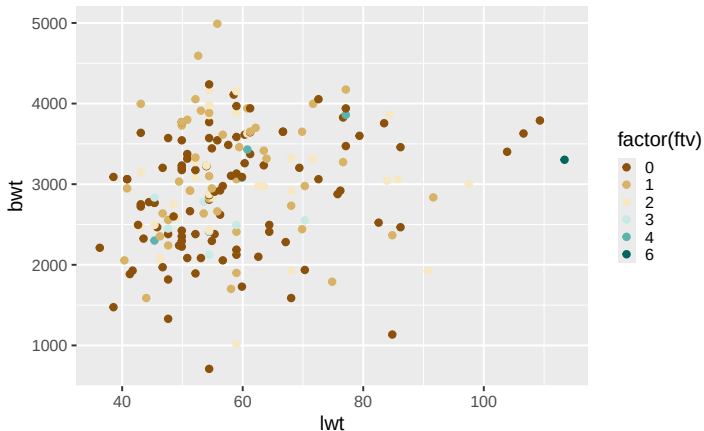
3. Para datos divergentes (cuando los datos tienen un punto medio significativo).

```
RColorBrewer::display.brewer.all(type = "div")
```



Ejemplo divergente

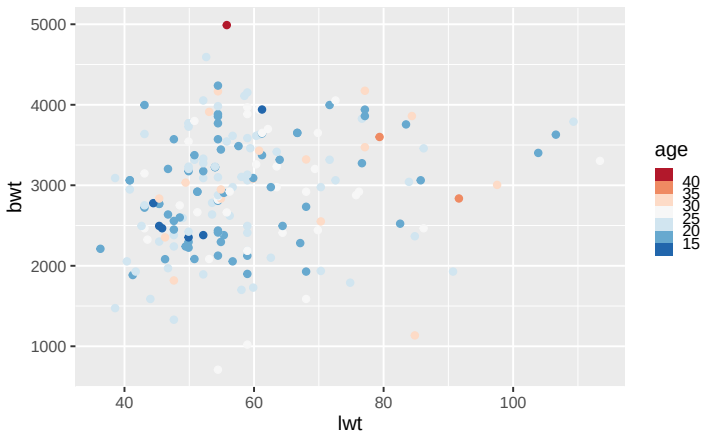
```
ggplot(bwt, aes(lwt,bwt, color=factor(ftv))) +  
  geom_point()+scale_colour_brewer(palette = "BrBG")
```



Paletas continuas

En el caso de trabajar con datos continuos hay que usar la instrucción `scale_colour_fermenter`

```
ggplot(bwt, aes(lwt,bwt, color=age)) + geom_point()+  
  scale_colour_fermenter(palette = "RdBu")
```



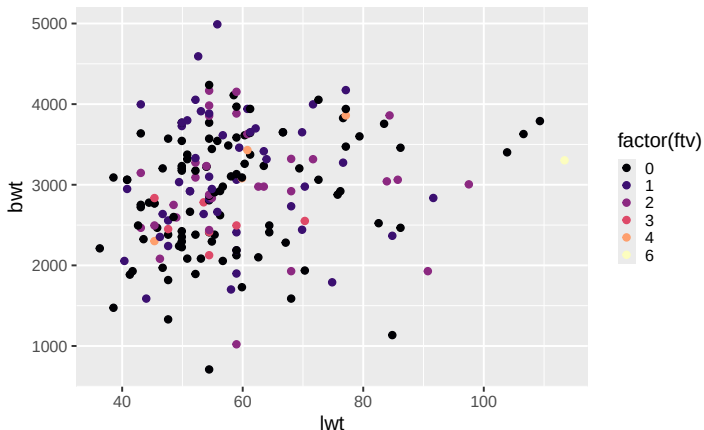
Viridis

Del paquete Viridis: A (magma), B (inferno), C (plasma), D (viridis), E (cividis), F (rocket), G (mako).

El paquete viridis organiza las paletas entre discretas y continuas en función de si se usa la instrucción `scale_colour_viridis_d` o `scale_colour_viridis_c` respectivamente.

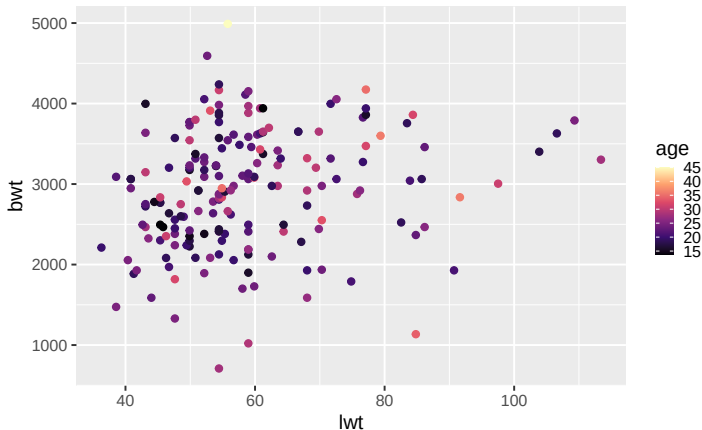
Ejemplo categórico/discreto

```
ggplot(bwt, aes(lwt,bwt, color=factor(ftv))) +  
  geom_point()+  
  scale_colour_viridis_d(option = "A")
```



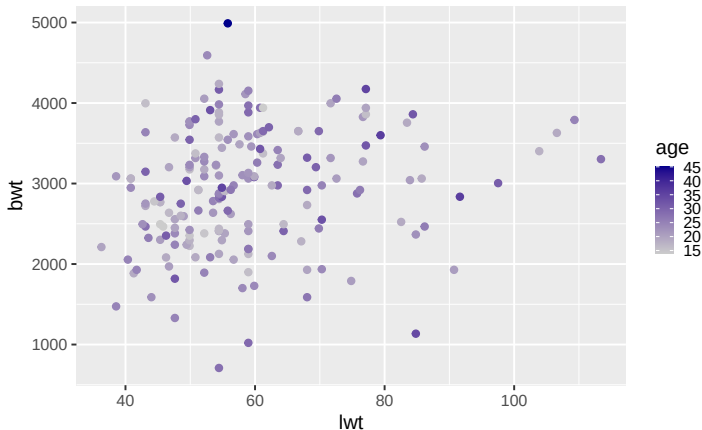
Ejemplo continuo

```
ggplot(bwt, aes(lwt,bwt, color=age)) + geom_point()+  
  scale_colour_viridis_c(option = "A")
```



A mano

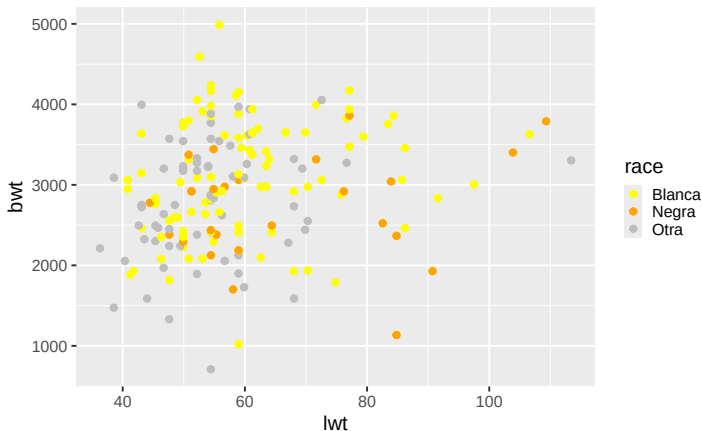
```
ggplot(bwt, aes(lwt,bwt, color=age)) + geom_point()+  
  scale_color_gradient(low = "grey80", high = "darkblue")
```



Puedes ver la selección en `colors()` o añadirlos en hex.

A mano

```
ggplot(bwt, aes(lwt,bwt, color=race)) + geom_point()+  
  scale_color_manual(values= c(Negra = "orange",  
    Blanca = "yellow", Otra = "gray"))
```

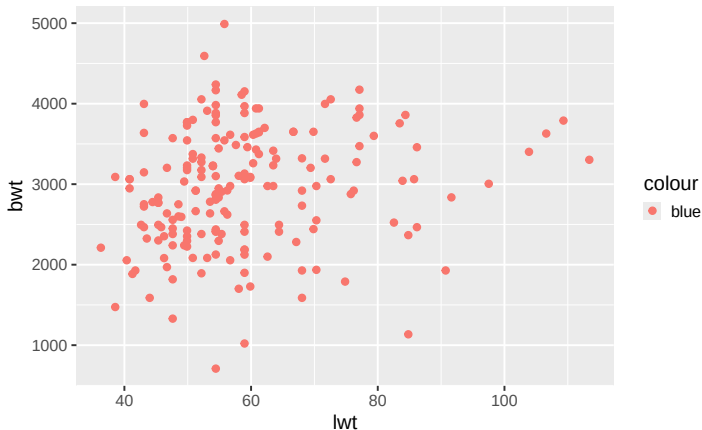


Color fijo

OJO: si queremos colorear en función de un dato, lo indicaremos dentro de aes, en cambio si queremos colorear todo igual lo haremos fuera de aes. Esto será así para cualquier parámetro que usemos.

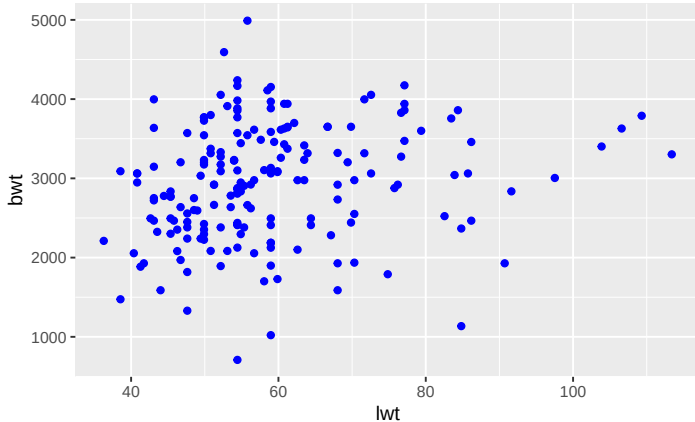
Ejemplo uso incorrecto color fijo

```
ggplot(bwt, aes(lwt,bwt)) +  
  geom_point(aes(color = "blue"))
```



Ejemplo uso correcto color fijo

```
ggplot(bwt, aes(lwt,bwt)) + geom_point(color = "blue")
```

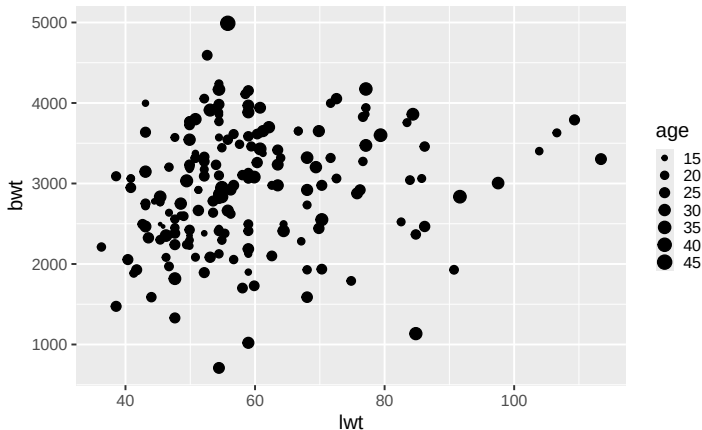


Tamaños

También podemos ajustar el tamaño de los puntos en función de una variable o de forma global.

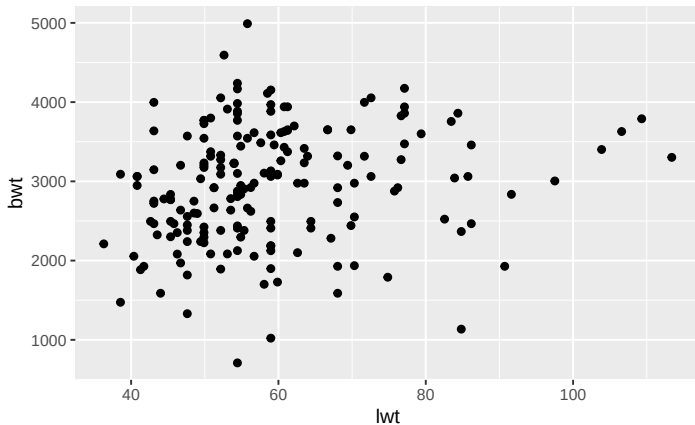
Ejemplo tamaño dependiente

```
ggplot(bwt, aes(lwt,bwt))+geom_point(aes(size = age))
```



Ejemplo tamaño fijo

```
ggplot(bwt, aes(lwt,bwt)) + geom_point(size = 3)
```

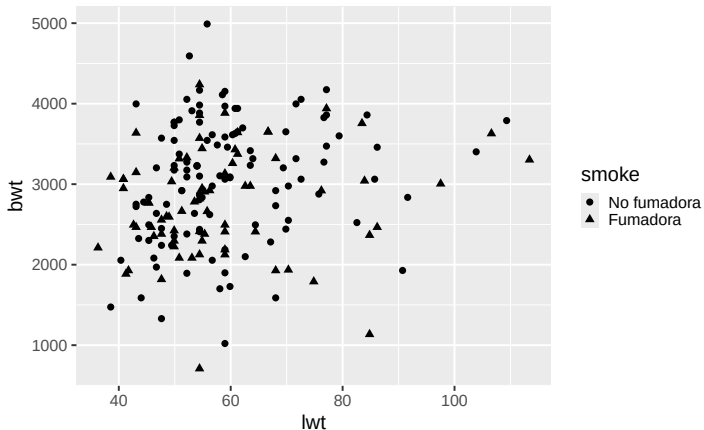


Formas

De la misma manera podemos asignar formas en función de una categoría o fijar una forma a todo el dibujo

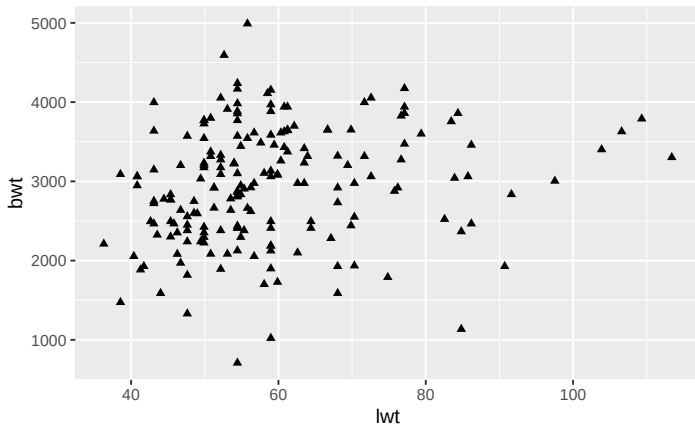
Forma dependiente

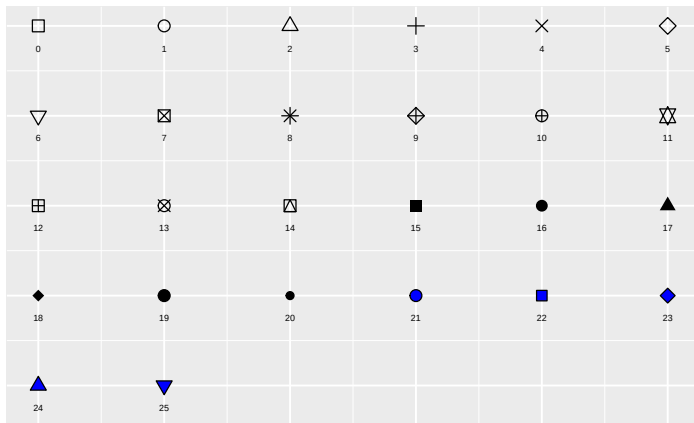
```
ggplot(bwt, aes(lwt,bwt)) +  
  geom_point(aes(shape = smoke))
```



Forma fija

```
ggplot(bwt, aes(lwt,bwt)) + geom_point(shape = 17)
```



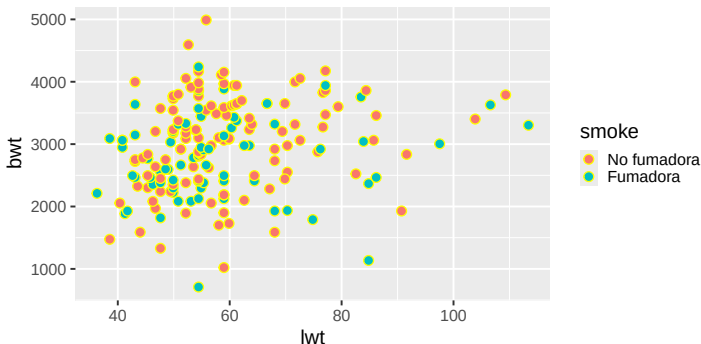


En el caso de usar más de 25 formas, ggplot2 usa caracteres ascii. Llegados a ese punto mejor dividir el dibujo en varios.

Bordes y fill

Podemos editar el borde de manera diferente al relleno usando `fill`, exclusivo de las figuras 21 hasta 25.

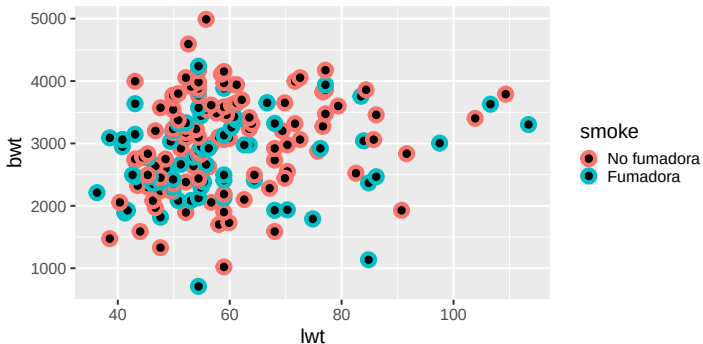
```
ggplot(bwt, aes(lwt,bwt)) +  
  geom_point(aes(fill = smoke),color = "yellow",  
             shape = 21,size = 4)
```



Bordes y stroke

También podemos alterar el grosor del borde usando `stroke`

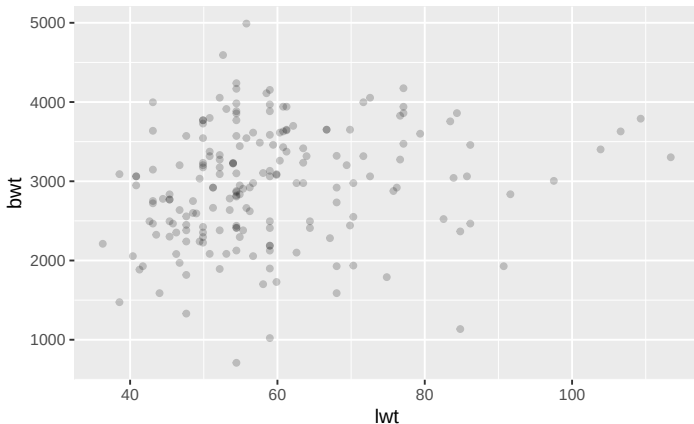
```
ggplot(bwt, aes(lwt,bwt)) +  
  geom_point(aes(color = smoke),fill = "black",  
             shape = 21,size = 4,stroke = 3)
```



Alpha

Cuando hay muchas figuras reducir la opacidad mejora la visualización.

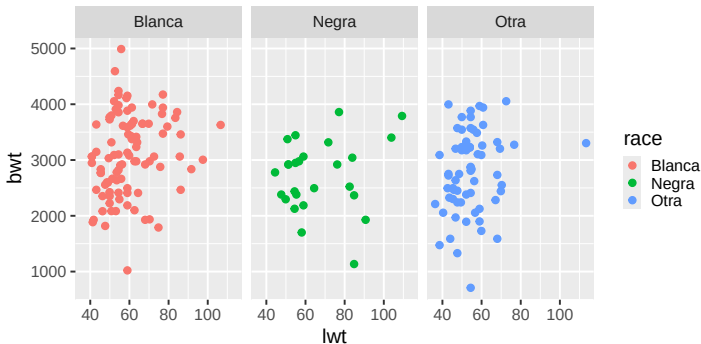
```
ggplot(bwt, aes(lwt,bwt)) +  
  geom_point(alpha = 0.2, size = 2.5)
```



Facetting

Para hacer un dibujo por categoría, basta añadir la instrucción `facet_wrap`, si queremos superponerlos, hacemos `group`.

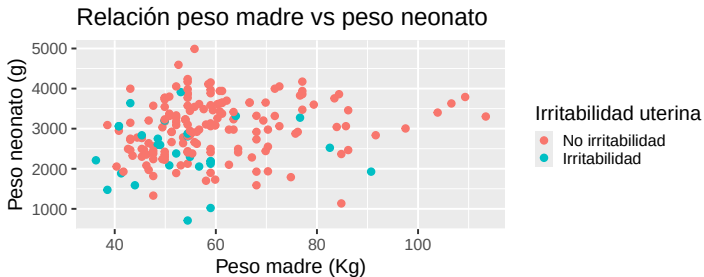
```
ggplot(bwt, aes(lwt,bwt)) +  
  geom_point(aes(color = race))+ facet_wrap(~race)
```



OJO: hay que añadir el `~` antes del nombre de la categoría.

Legenda y ejes

```
ggplot(bwt, aes(lwt,bwt)) + geom_point(aes(color=ui))+  
  labs(y= "Peso neonato (g)",  
        x="Peso madre (Kg)",  
        title = "Relación peso madre vs peso neonato",  
        color = "Irritabilidad uterina")
```

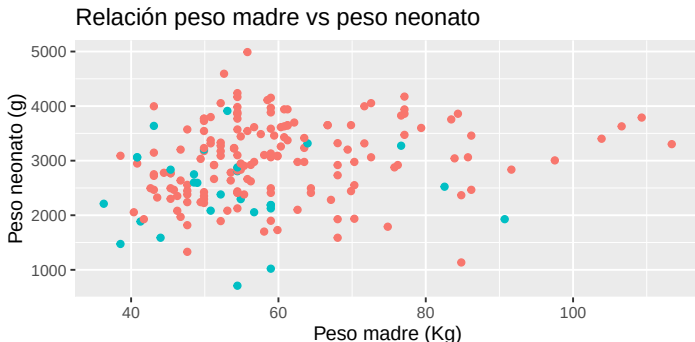


Para ajustar el nombre de la leyenda ponemos el nombre del parámetro que se está usando para que aparezca.

Quitar leyenda

También podemos eliminar la leyenda completamente.

```
ggplot(bwt, aes(lwt,bwt)) +  
  geom_point(aes(color=ui),show.legend = FALSE)+  
  labs(y= "Peso neonato (g)",  
       x="Peso madre (Kg)",  
       title = "Relación peso madre vs peso neonato")
```

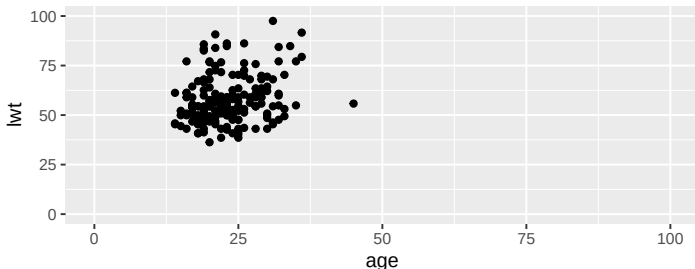


Escalas

Es posible manipular manualmente el tamaño de los ejes.

```
ggplot(bwt,aes(age,lwt))+  
  geom_point()+  
  xlim(0,100)+  
  ylim(0,100)
```

Warning: Removed 4 rows containing missing values or values
(`geom\_point()`).

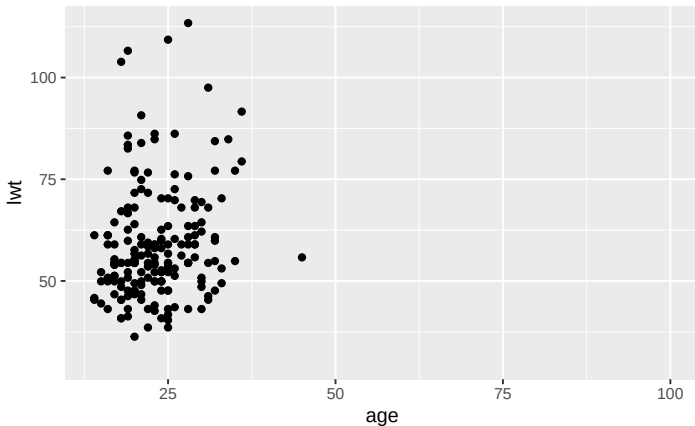


En este caso R nos da el aviso de hay puntos que no están

Elegir solo uno de los extremos

Podemos usar NA para que R decidá por nosotros el valor.

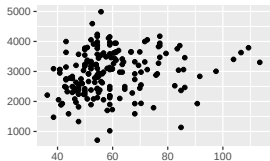
```
ggplot(bwt,aes(age,lwt))+  
  geom_point()+  
  xlim(NA,100)+  
  ylim(30,NA)
```



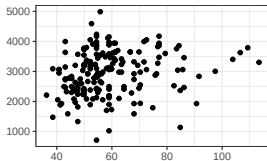
Temas

Podemos elegir entre varios temas.

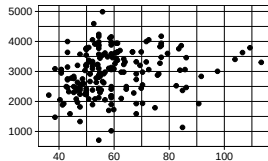
+ theme_gray()



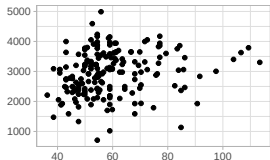
+ theme_bw()



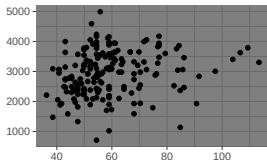
+ theme_linedraw()



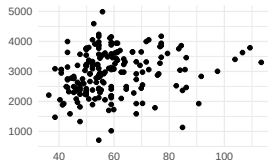
+ theme_light()



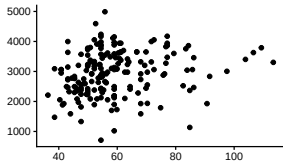
+ theme_dark()



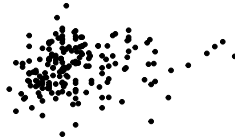
+ theme_minimal()



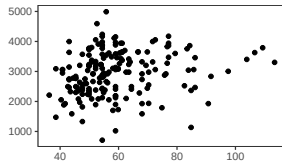
+ theme_classic()



+ theme_void()



+ theme_test()

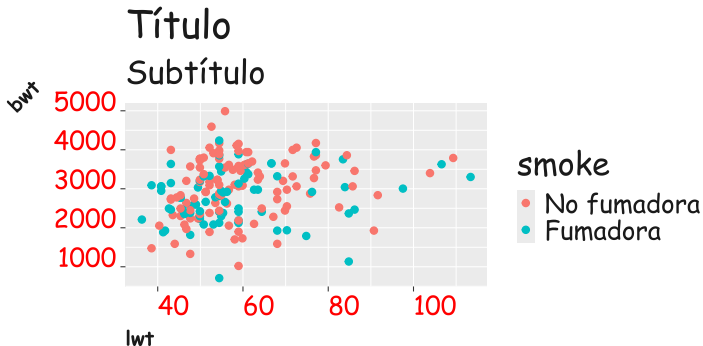


Creación de un tema personalizado

```
theme_everything <- theme(  
  line = element_line(colour = "gray20",  
                        linewidth = 0.5, linetype = 1,  
                        lineend = "butt"),  
  rect = element_rect(fill = "white", colour = NA,  
                       linewidth = 0.5, linetype = 1),  
  text = element_text(family = "comicsans", face = "plain",  
                       colour = "gray10", size = 32,  
                       lineheight = 0.9, hjust = 0,  
                       vjust = 0, margin = margin(),  
                       angle = 0),  
  axis.title = element_text(face = "bold", size = 20),  
  axis.title.x = element_text(margin = margin(t = 8)),  
  axis.title.y = element_text(margin = margin(r = 8),  
                              angle = 45),  
  axis.text = element_text(colour = "red"))
```


Ejemplo de uso del tema personalizado

```
ggplot(bwt, aes(lwt, bwt)) +  
  geom_point(aes(color = smoke)) +  
  theme_everything +  
  labs(title = "Título", subtitle = "Subtítulo")
```



Añadir fuentes

Con el paquete `showtext` podemos añadir fuentes de nuestro sistema (solo para windows).

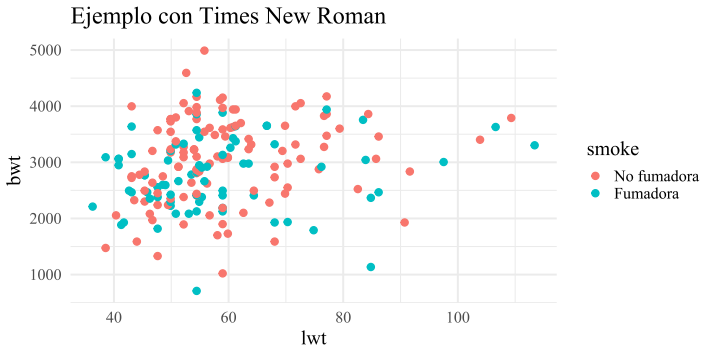
```
library(showtext)
font_add(family = "Times New Roman",
         regular = "C:/Windows/Fonts/times.ttf",
         bold    = "C:/Windows/Fonts/timesbd.ttf",
         italic  = "C:/Windows/Fonts/timesi.ttf",
         bolditalic = "C:/Windows/Fonts/timesbi.ttf")

font_add(family = "comicsans",
         regular = "C:/Windows/Fonts/comic.ttf",
         bold    = "C:/Windows/Fonts/comicbd.ttf",
         italic  = "C:/Windows/Fonts/comici.ttf",
         bolditalic = "C:/Windows/Fonts/comicz.ttf")

showtext_auto() # activa el renderizado de showtext
```

Ejemplo fuente sobre tema ya existente

```
ggplot(bwt,aes(lwt,bwt))+  
  geom_point(aes(color = smoke))+  
  labs(title = "Ejemplo con Times New Roman") +  
  theme_minimal(base_family = "Times New Roman",  
    base_size = 20)
```



Geometría

Geometría

Ahora que ya conocemos las opciones de personalización, estudiamos las opciones de geometría disponibles.

La mejor manera de acceder a la lista de geometrías es usando la “chuleta” de ggplot2 [<https://posit.co/wp-content/uploads/2022/10/data-visualization-1.pdf>]

Cuándo usar cada geometría depende del tipo de datos y de qué se quiera explicar. Vamos a ver algunas de las más típicas ¡Se pueden combinar!

Interpolación de valores

Datos temporales suelen representarse con una poligonal que interpola los puntos.

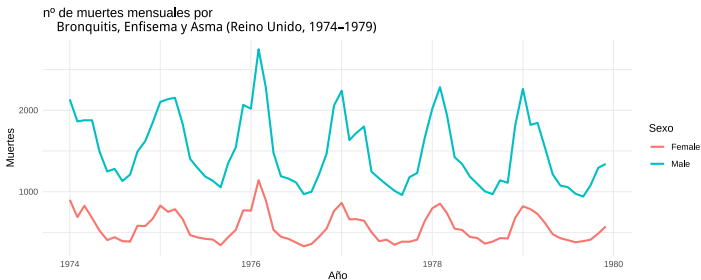
Para este ejemplo usamos datos temporales (mdeaths,fdeaths) contenidos en R base.

```
df <- data.frame(  
  date    = seq(as.Date("1974-01-01"), by = "month",  
                length.out = length(mdeaths)),  
  Male    = as.numeric(mdeaths),  
  Female  = as.numeric(fdeaths)  
) |>  
  pivot_longer(cols = c(Male, Female),  
               names_to = "sex", values_to = "deaths")
```

Linewidth

El grosor se puede ajustar con linewidth.

```
ggplot(df, aes(x = date, y = deaths, color = sex)) +  
  geom_line(linewidth = 1) +  
  labs(title = "nº de muertes mensuales por  
    Bronquitis, Enfisema y Asma (Reino Unido, 1974-1979)",  
    x = "Año", y = "Muertes", color = "Sexo")  
  ) + theme_minimal()
```

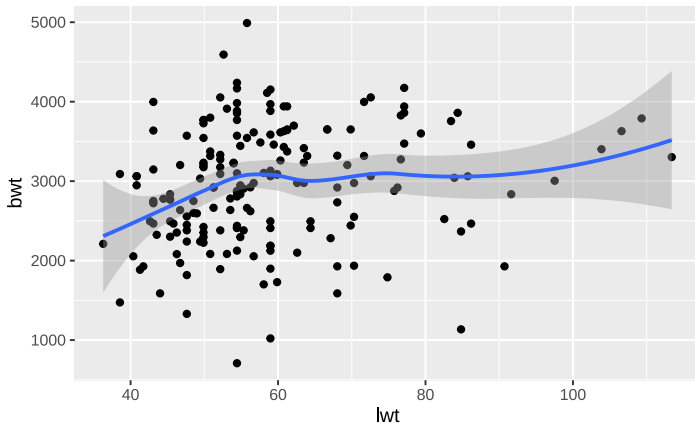


OJO: Observad como, al añadir un color, nos ha separado dos datos en dos curvas diferentes.

Curva de los datos

Para ajustar los datos a una curva de regresión.

```
ggplot(bwt, aes(lwt,bwt))+  
  geom_point()+  
  geom_smooth()
```



Regresión lineal

```
ggplot(bwt, aes(lwt,bwt))+  
  geom_point()+  
  geom_smooth(method = "lm")
```

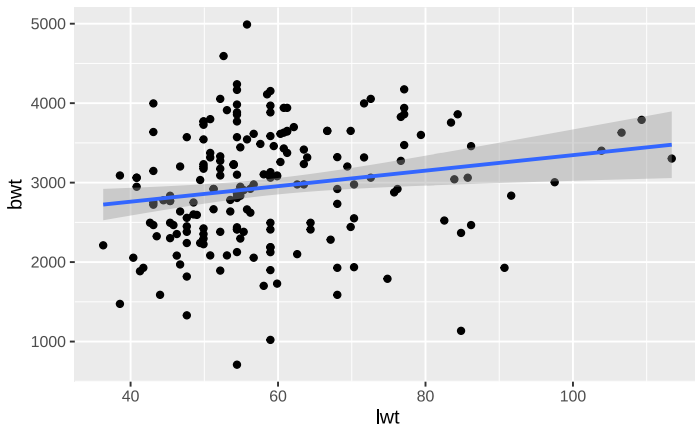
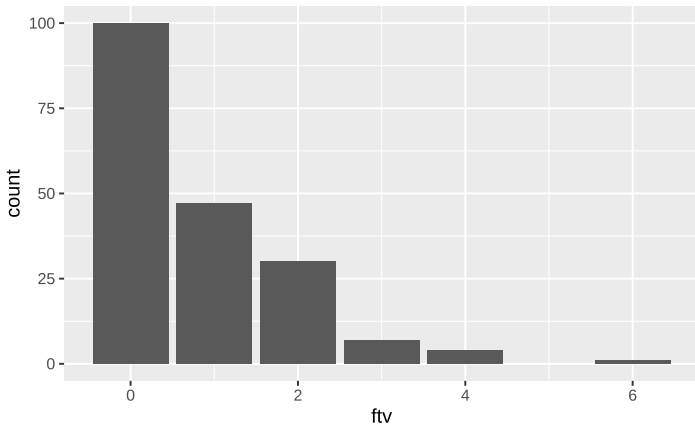


Gráfico de barras

Para mostrar la distribución de variables categoricas.

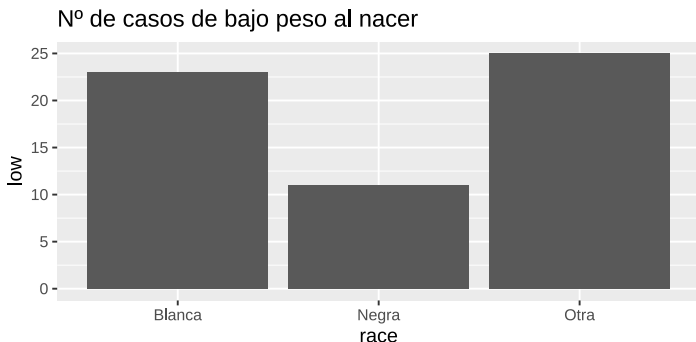
```
ggplot(bwt, aes(ftv))+  
  geom_bar()
```



geom_col()

Si queremos reflejar una magnitud respecto de otra por categoría hay que usar geom_col.

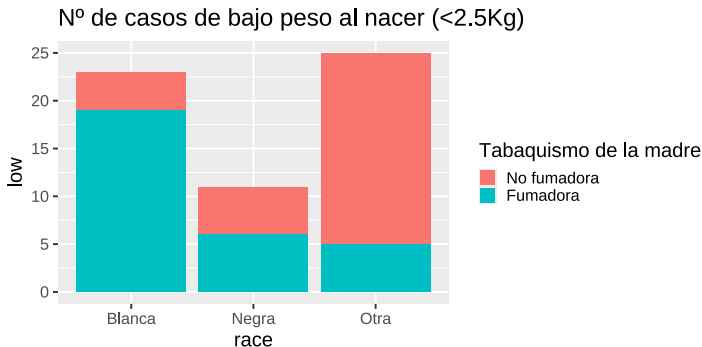
```
ggplot(bwt, aes(race, low)) +  
  geom_col() +  
  labs(title = "Nº de casos de bajo peso al nacer")
```



Recuerda lo aprendido

Recordad que podemos aplicar todas las técnicas anteriores.

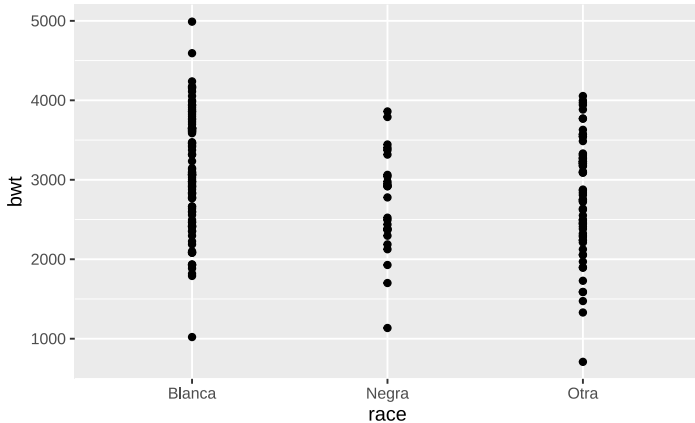
```
ggplot(bwt, aes(race, low)) +  
  geom_col(aes(fill = factor(smoke))) +  
  labs(title = "Nº de casos de bajo peso al nacer (<2.5Kg)",  
        fill = "Tabaquismo de la madre")
```



Boxplots y jittered points

Nos pueden aparecer casos de esta forma.

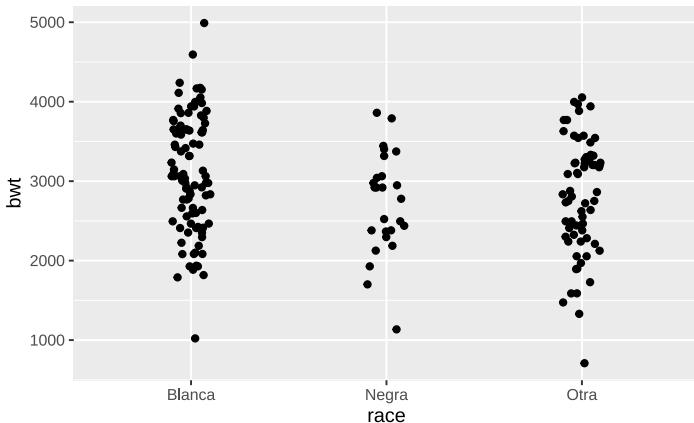
```
ggplot(bwt,aes(race,bwt)) + geom_point()
```



geom_jitter

Para facilitar la visualización de los datos tenemos varias opciones. La primera es mover ligeramente los puntos con `geom_jitter`

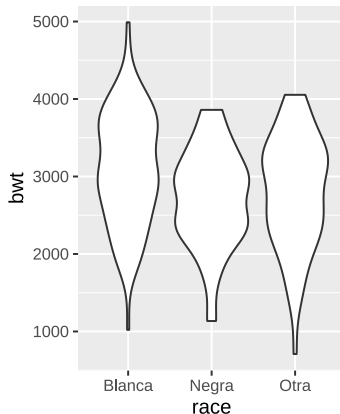
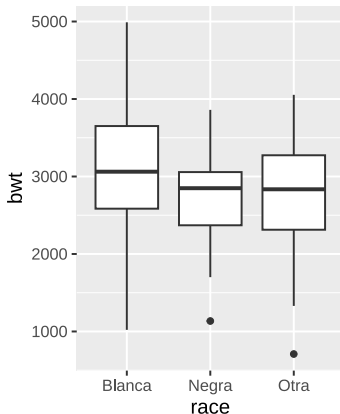
```
ggplot(bwt, aes(race, bwt)) +  
  geom_jitter(width = 0.1)
```



Boxplot y violin plot

Otra alternativa es resumir los datos.

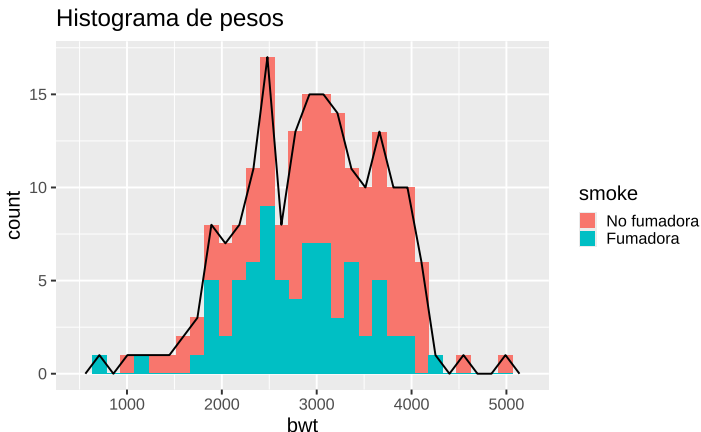
```
v1<- ggplot(bwt,aes(race,bwt))+geom_boxplot()  
v2 <- ggplot(bwt,aes(race,bwt))+geom_violin()  
v1|v2
```



Histogramas y polígonos de frecuencias

Un histograma, o un polígono de frecuencias, nos permite ver la distribución de los datos.

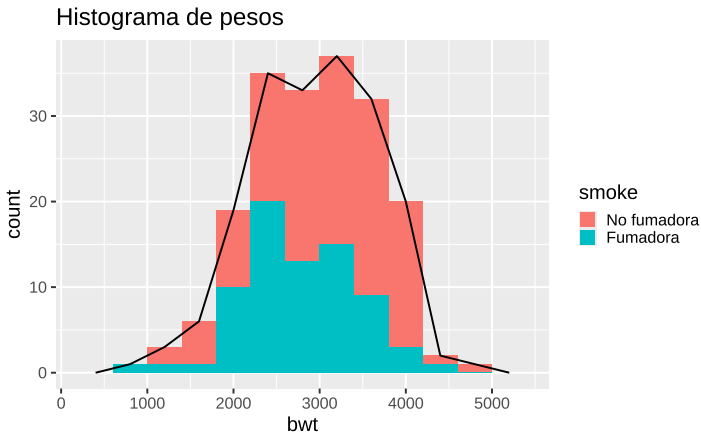
```
ggplot(bwt,aes(bwt))+ geom_histogram(aes(fill=smoke))+  
  geom_freqpoly()+labs(title = "Histograma de pesos")
```



Ajustar el n° de bind

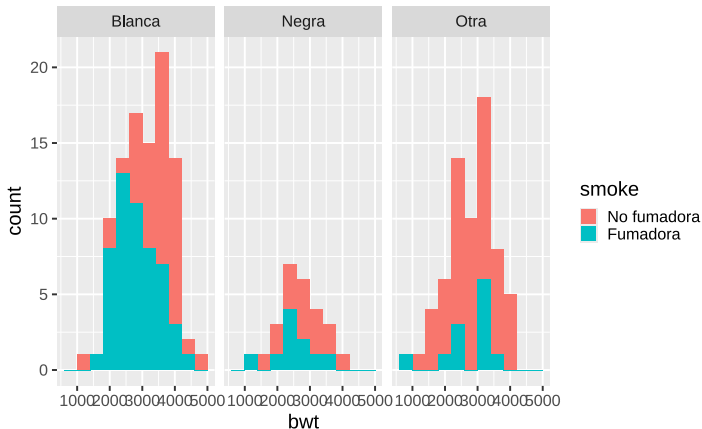
Por defecto siempre genera 30 columnas. Binwidth que nos indica el tamaño de cada grupo y bins fija el n° total.

```
ggplot(bwt,aes(bwt))+ geom_histogram(aes(fill=smoke),binwidth=400,
  geom_freqpoly(binwidth=400))+labs(title = "Histograma de pesos")
```



Combinando con facet_wrap

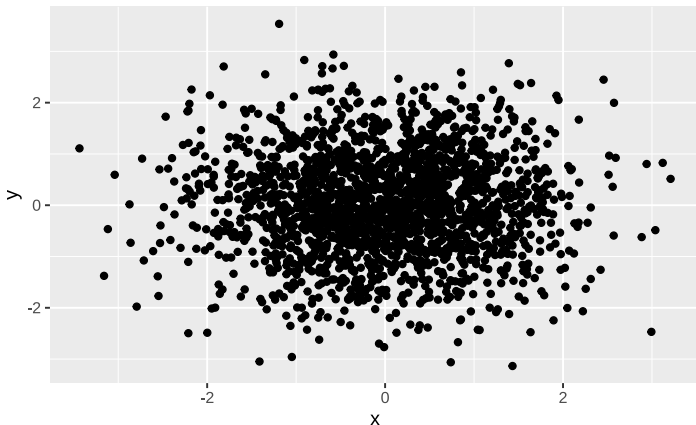
```
ggplot(bwt, aes(bwt)) +  
  geom_histogram(aes(fill=smoke),  
    binwidth = 400) + facet_wrap(~race)
```



Geom_bin2d y geom_hex

Cuando hay muchos datos fácil acabar con problemas de overplotting, esto es hacer el gráfico difícil de leer.

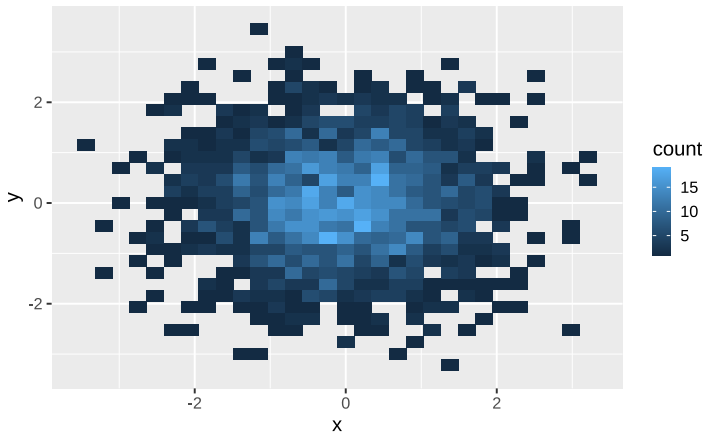
```
df <- data.frame(x = rnorm(2000), y = rnorm(2000))  
norm <- ggplot(df, aes(x, y))  
norm + geom_point()
```



Mapa de calor

Podemos representarlo con un gráfico de calor

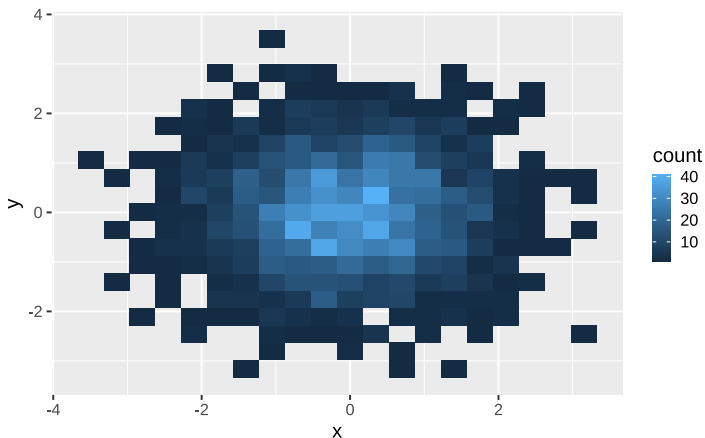
```
norm+geom_bin2d()
```



Ajustar el nº de bins

Recordad que esto funciona como un `geom_bar`, hay que indicar bins o binwidth para ajustar a nuestros datos

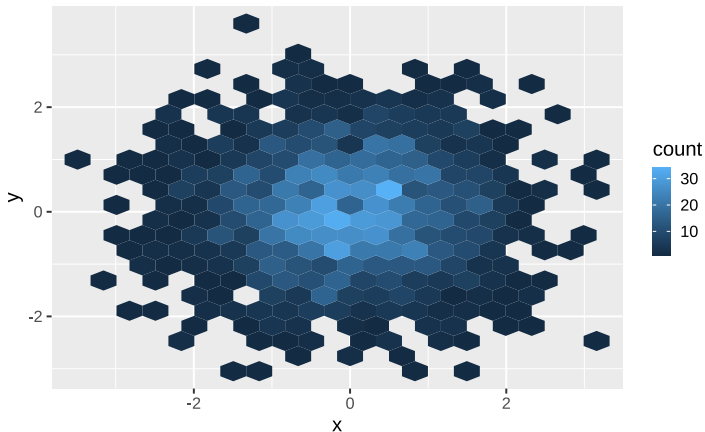
```
norm+geom_bin2d(bins= 20)
```



geom_hex

Alternativamente, se puede usar el paquete `hexbin` para producir hexágonos.

```
library(hexbin)
d<- norm + geom_hex(bins=20)
d
```



Y mucho más

Si queremos generar mapas tenemos la función `geom_sf` que usa datos `sf` (Simple Features) para representar objetos geométricos. Podemos entender `sf` como un data frame de mapas

```
library(sf)           #para trabajar con mapas
library(rnaturalearth)#para descargar mapas
library(ggthemes)     #más colores
```

Preparación datos mapa

```
#Datos España
es_ccaa <- ne_states(country = "Spain",
returnclass = "sf")
# Datos baleares
balears <- subset(es_ccaa, grepl("Baleares", name,
ignore.case = TRUE))
```


Ejemplo mapa

```
a<- ggplot(balears) +  
  geom_sf(fill="grey90", color = "grey20",  
          linewidth = 0.4) + coord_sf(expand = 0) +  
  labs(title = "Illes Balears") +  
  theme_void(base_size = 12)  
a
```

Illes Balears



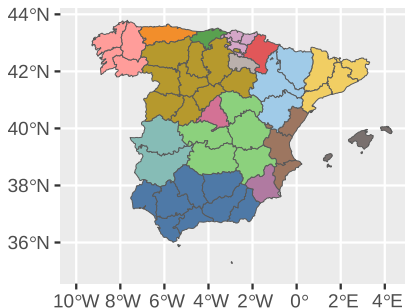
Creación paleta para España

```
# Creación a mano de paleta
lv <- sort(unique(es_ccaa$ccaa))
pal <- ggthemes::tableau_color_pal("Tableau 20")(19)
names(pal) <- lv
```

Ejemplo mapa 2

```
b<- ggplot(es_ccaa, aes(fill=factor(region)))+  
  geom_sf()+  
  coord_sf(xlim=c(-10,NA),  
           ylim=c(35,NA))+  
  guides(fill = guide_legend(ncol = 2, byrow = TRUE))+  
  labs(title= "Mapa de España",fill= "Comunidad Autónoma")-  
  scale_fill_manual(values = pal)
```

Mapa de España



Comunidad Autónoma

Andalucía	Aragón
Asturias	Canary Is.
Cantabria	Castilla-La Mancha
Castilla y León	Cataluña
Ceuta	Extremadura
Foral de Navarra	Galicia
Islas Baleares	La Rioja
Madrid	Melilla
Murcia	País Vasco
Valenciana	

Manipulación de archivos y aritmética de ggplot

Guardar gráficos

Una vez hemos hecho un gráfico podemos guardarlo de la siguiente manera

```
p <- ggplot(bwt,aes(age,bwt)) + geom_point()
# Si queremos png o jpeg
ggsave("plot.png",p,
        width = 5,
        height = 5,
        units = "cm",
        dpi = 300)
# En el caso de pdf, svg (mejor para publicaciones)
ggsave("plot.pdf",p,
        width = 5,
        height = 5,
        units = "cm")
```

Los formatos pdf y svg guardan la estructura interna lo que hace que siempre se vean bien independiente del tamaño, en cambio un png o jpeg es un mapa de bits y la resolución depende de los dpi.

Aritmética de ggplot

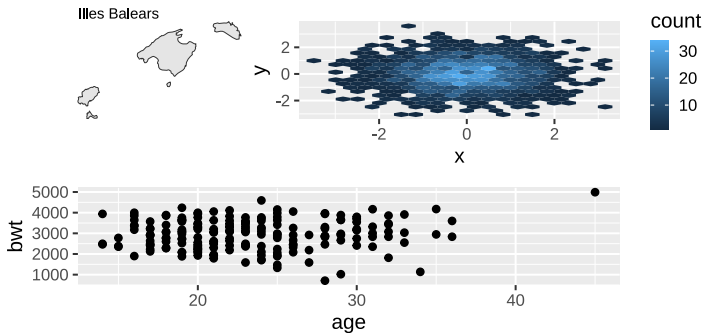
El paquete ggplot2 se ha planteado con una aritmética interna para facilitar hacer muchas gráficas con el mismo conjunto de datos.

```
c <- ggplot(bwt,aes(age,bwt)) # guardamos  
  
c <- c + geom_point() #la completamos
```

patchwork

El paquete `patchwork` permite combinar varios gráficos de manera fácil.

(a | d) / c



Parámetros de patchwork

Si queremos agrupar las leyendas en un lado podemos hacerlo con `plot_layout`.

```
(a + d + c) + plot_annotation(title = "Varios") +  
  plot_layout(guides = 'collect')
```

Varios

Illes Balears

